UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE

PRUEBAS DE SOFTWARE

Análisis, Diseño e Implementación del Sistema Web de Gestión Bibliotecaria Universitaria

INTEGRANTES

Danny Ayuquina
Lucas Góngora
David Pilaguano

FECHA: 2026/05/14

ÍNDICE

1. Introducción.... 4
1.1 Objetivo General.... 4
1.2 Objetivos Específicos.... 4
2. Alcance del Proyecto.... 5
2.1. Funcionalidades incluidas.... 5
2.2. Restricciones del Proyecto.... 5
2.3 Restricciones Técnicas.... 5
3. Metodologías de desarrollo.... 6
3.1 Domain Driven Design (DDD).... 6
4. Requisitos e Historias de usuario del sistema.... 6
4.1 Requisitos Funcionales.... 6
RF-00 Autenticación y Acceso al Sistema.... 6
RF-01 Gestión de Usuarios del Sistema.... 6
RF-02 Gestión de Usuarios Bibliotecarios.... 7
RF-03 Gestión de Roles de Usuario.... 7
RF-04 Gestión de Libros.... 7
RF-05 Gestión de Ejemplares.... 7
RF-06 Gestión de Préstamos.... 7
RF-07 Gestión de Reservas.... 8
RF-08 Gestión de Sanciones.... 8
4.2 Requisitos No Funcionales.... 8
RNF-01 Seguridad.... 8
RNF-02 Integridad.... 8
RNF-03 Escalabilidad.... 8
RNF-04 Mantenibilidad.... 8
RNF-05 Rendimiento.... 8
RNF-06 Calidad.... 8
4.3 Historias de Usuario Críticas.... 9
HU-01 — Inicio de Sesión Administrativo.... 9
HU-02 — Registrar préstamo válido.... 9
HU-03 — Bloquear préstamo de ejemplar no disponible.... 9
HU-04 — Validar límite máximo de préstamos.... 10
HU-05 — Registrar devolución de ejemplar.... 10
HU-06 — Generar sanción por devolución tardía.... 10
HU-07 — Registrar reservas.... 11
HU-08 — Registrar ejemplares físicos.... 11
5. Diseño de Base de Datos.... 11
5.1 Entidades.... 11
usuario_sistema.... 11
usuario.... 13
rol_usuario.... 14
libro.... 15
categoria.... 16

editorial....16
autor....17
libro_autor....18
ejemplar....18
prestamo....19
sancion....20
reserva....22
auditoria....23
5.2 Relaciones....24
Tabla de relaciones....24
5.3 Modelado....26
5.4 Script....26
5.5 Reglas de negocio implementadas....36
6. Arquitectura General del Sistema....37
6.1 Arquitectura Global....37
6.2. Arquitectura Backend....37
Controllers....38
Services....38
DTOs....38
Entities....39
6.3. Arquitectura Frontend....39
7. Testing y Calidad del Software....39
7.1. Estrategia de Calidad....39
7.2. Jest....40
Pruebas Unitarias....40
7.3. Supertest....40
7.4. SonarQube....40
7.5. ESLint....41
8. Seguridad y Validaciones....41
9. Tradeoffs Técnicos....41
9.1. Uso de Supabase....41
9.2. Arquitectura Monolítica Modular....41
10. Conclusiones....42
11. Recomendaciones....43

1. Introducción

El presente documento técnico describe el análisis, diseño, arquitectura, estructura tecnológica y lineamientos de calidad implementados para el desarrollo del sistema web de gestión de biblioteca universitaria.

El proyecto tiene como finalidad digitalizar y controlar los procesos operativos relacionados con la administración bibliotecaria, permitiendo gestionar préstamos, devoluciones, reservas, sanciones, ejemplares físicos y auditoría de operaciones realizadas dentro del sistema.

A diferencia de aplicaciones CRUD básicas, el sistema busca representar reglas reales del dominio bibliotecario universitario, incorporando restricciones, validaciones y reglas de negocio alineadas con escenarios operativos reales.

El sistema se desarrolla bajo una arquitectura full-stack desacoplada, utilizando tecnologías modernas tanto para frontend como backend, así como un enfoque basado en API REST y persistencia de datos sobre PostgreSQL mediante Supabase.

1.1 Objetivo General

Desarrollar un sistema web de gestión bibliotecaria universitaria capaz de administrar libros, ejemplares físicos, préstamos, reservas, sanciones y auditorías o gestiones realizadas en la biblioteca mediante una arquitectura moderna, modular y escalable basada en tecnologías web actuales.

1.2 Objetivos Específicos

- Modelar correctamente las entidades y relaciones del dominio bibliotecario.
- Implementar reglas de negocio mediante restricciones y validaciones en la base de datos como en los códigos para el Backend y el Frontend.
- Desarrollar un backend modular usando NestJS.
- Implementar una interfaz moderna utilizando Next.js y React.
- Incorporar pruebas automatizadas unitarias y E2E para validar reglas de negocio, comportamiento de endpoints y funcionamiento general del sistema.
- Aplicar herramientas de análisis estático y verificación de código mediante ESLint y SonarQube para mejorar la mantenibilidad, calidad y detección temprana de errores.
- Mantener una estructura organizada y desacoplada entre frontend, backend, pruebas y documentación técnica del proyecto.

2. Alcance del Proyecto

2.1. Funcionalidades incluidas

El sistema contempla las siguientes funcionalidades:

- Registro y gestión de libros.
- Gestión de ejemplares físicos.
- Registro y administración de usuarios bibliotecarios.
- Administración de usuarios del sistema.
- Gestión de préstamos.
- Registro de devoluciones.
- Gestión de reservas.
- Administración de sanciones.
- Auditoría básica de operaciones.

2.2. Restricciones del Proyecto

El sistema **NO** contempla:

- Aplicación móvil.
- Integración bancaria.
- Pagos en línea.
- Gestión de ebooks.
- Recomendaciones automáticas.
- Integración con sistemas externos universitarios.
- Arquitectura de microservicios.
- Procesamiento distribuido.
- Sistema de autenticación federada.

2.3 Restricciones Técnicas

- El proyecto utiliza PostgreSQL administrado mediante Supabase.
- El backend se implementa únicamente con NestJS.
- El frontend utiliza Next.js con App Router.
- El lenguaje principal es TypeScript.
- Las pruebas automatizadas se implementan con Jest y Supertest.
- El análisis estático de código se realiza con ESLint y SonarQube.
- La arquitectura sigue un enfoque modular.

3. Metodologías de desarrollo

3.1 Domain Driven Design (DDD)

El diseño orientado al dominio permite representar correctamente las reglas reales del sistema bibliotecario.

Se prioriza:

- modelado correcto de entidades,
- relaciones consistentes,
- restricciones del dominio,
- separación entre lógica de negocio y persistencia

4. Requisitos e Historias de usuario del sistema

4.1 Requisitos Funcionales

RF-00 Autenticación y Acceso al Sistema

El sistema debe permitir:

- autenticación de usuarios administrativos,
- validación de credenciales,
- control de acceso según rol del sistema,
- restricción de acceso a usuarios bloqueados o inactivos.

El acceso al sistema estará limitado a:

- administradores,
- bibliotecarios.

RF-01 Gestión de Usuarios del Sistema

El sistema debe permitir:

- registrar usuarios administrativos,
- administrar credenciales,
- asignar roles del sistema,
- controlar estados de acceso,
- bloquear usuarios internos.

Roles contemplados:

- ADMINISTRADOR,
- BIBLIOTECARIO.

Estados permitidos:

- ACTIVO,
- INACTIVO,
- BLOQUEADO.

RF-02 Gestión de Usuarios Bibliotecarios

El sistema debe permitir registrar usuarios bibliotecarios con:

- información personal,
- correo institucional,
- estado,
- rol de préstamo.

RF-03 Gestión de Roles de Usuario

El sistema debe permitir definir:

- máximo de préstamos,
- días permitidos de préstamo,
- restricciones institucionales.

RF-04 Gestión de Libros

El sistema debe permitir:

- registrar libros,
- asociar categorías,
- asociar editoriales,
- asociar autores.

RF-05 Gestión de Ejemplares

El sistema debe permitir:

- registrar ejemplares físicos,
- controlar disponibilidad,
- registrar ubicación física,
- administrar estados.

RF-06 Gestión de Préstamos

El sistema debe permitir:

- registrar préstamos,
- controlar fechas,

- registrar devoluciones,
- validar disponibilidad,
- validar límites institucionales.

RF-07 Gestión de Reservas

El sistema debe permitir reservar libros cuando no existan ejemplares disponibles.

RF-08 Gestión de Sanciones

El sistema debe permitir:

- registrar penalizaciones,
- administrar montos,
- controlar estados,
- relacionar sanciones con préstamos.

4.2 Requisitos No Funcionales

RNF-01 Seguridad

Las contraseñas deben almacenarse cifradas mediante hash.

RNF-02 Integridad

El sistema debe mantener integridad referencial mediante claves foráneas.

RNF-03 Escalabilidad

La arquitectura debe permitir ampliaciones futuras.

RNF-04 Mantenibilidad

El sistema debe mantener separación modular entre componentes.

RNF-05 Rendimiento

Las consultas frecuentes deben optimizarse mediante índices.

RNF-06 Calidad

El sistema debe incorporar:

- pruebas automatizadas,
- análisis estático.

4.3 Historias de Usuario Críticas

Las siguientes historias de usuario representan los procesos funcionales más importantes del sistema bibliotecario y permiten definir las validaciones operativas, restricciones institucionales y reglas de negocio implementadas tanto en base de datos como en backend.

Las historias de usuario sirven como referencia para:

- validaciones funcionales,
- definición de reglas de negocio,
- pruebas automatizadas,
- validaciones del sistema,
- control de integridad operativa.

HU-01 — Inicio de Sesión Administrativo

Descripción
Como usuario administrativo, quiero iniciar sesión en el sistema para acceder a las funcionalidades bibliotecarias según mis permisos.

Criterios de aceptación

- el sistema debe validar credenciales,
- el usuario debe encontrarse ACTIVO,
- usuarios BLOQUEADOS no pueden ingresar,
- el acceso debe depender del rol del sistema,
- las contraseñas deben almacenarse cifradas.

HU-02 — Registrar préstamo válido

Descripción
Como bibliotecario, quiero registrar préstamos de ejemplares disponibles para permitir circulación bibliográfica.

Criterios de aceptación

- el usuario debe encontrarse ACTIVO,
- el ejemplar debe estar DISPONIBLE,
- debe generarse fecha límite automáticamente,
- el préstamo debe registrarse en estado ACTIVO,
- el ejemplar cambiar automáticamente a PRESTADO.

HU-03 — Bloquear préstamo de ejemplar no disponible

Descripción
Como sistema, quiero impedir préstamos sobre ejemplares no disponibles para mantener integridad del inventario.

Criterios de aceptación

- no permitir préstamos sobre ejemplares:
  - PRESTADO,
  - PERDIDO,
  - DANIADO,
  - MANTENIMIENTO.
- el sistema debe rechazar la operación.

HU-04 — Validar límite máximo de préstamos

Descripción
Como sistema, quiero validar límites máximos de préstamos según el rol institucional del usuario.

Criterios de aceptación

- estudiantes:
  - máximo 3 préstamos.
- docentes:
  - máximo 5 préstamos.
- administrativos:
  - máximo configurable.
- el sistema debe impedir exceder el límite permitido.

HU-05 — Registrar devolución de ejemplar

Descripción
Como bibliotecario, quiero registrar devoluciones para actualizar la disponibilidad de inventario.

Criterios de aceptación

- el préstamo cambia a DEVUELTO,
- el ejemplar cambia a DISPONIBLE,
- debe registrarse fecha devolución real.

HU-06 — Generar sanción por devolución tardía

Descripción
Como sistema, quiero generar sanciones automáticas cuando exista retraso en la devolución de ejemplares.

Criterios de aceptación

- si la devolución supera la fecha límite:
    - generar sanción,
    - asociar sanción al préstamo,
    - registrar estado de la sanción.

HU-07 — Registrar reservas

Descripción
Como usuario, quiero reservar libros sin disponibilidad inmediata para obtener prioridad futura.

Criterios de aceptación

- solo permitir reservas cuando no existan ejemplares DISPONIBLES,
- la reserva debe quedar ACTIVA,
- debe registrarse fecha de expiración,
- usuarios BLOQUEADOS no pueden reservar.

HU-08 — Registrar ejemplares físicos

Descripción
Como bibliotecario, quiero registrar ejemplares físicos para mantener el control del inventario institucional.

Criterios de aceptación

- cada ejemplar debe poseer código único,
- el ejemplar debe asociarse a un libro,
- el estado inicial debe ser DISPONIBLE.

5. Diseño de Base de Datos

5.1 Entidades

usuario_sistema

Justificación: Representa a las personas autorizadas para acceder al software de gestión bibliotecaria.

Ejemplos:

- bibliotecarios
- administradores

Campos:

Campo	Tipo	Justificación
id_usuario_sistema	BIGINT	Identificador interno
username	VARCHAR(50)	Nombre de acceso
password_hash	VARCHAR(255)	Contraseña cifrada, Las contraseñas no se almacenan en texto plano por seguridad.
primer_nombre	VARCHAR(50)	Primer nombre del usuario_sistema
segundo_nombre	VARCHAR(50)	Segundo nombre del usuario_sistema
primer_apellido	VARCHAR(50)	Primero apellido del usuario_sistema
segundo_apellido	VARCHAR(50)	Segundo apellido del usuario_sistema
estado	VARCHAR(20)	Control de acceso
rol_sistema	VARCHAR(30)	Permisos dentro del sistema

usuario

Justificación: Representa a las personas que pueden solicitar préstamos dentro de la biblioteca.

Ejemplos:

- estudiantes
- docentes
- administrativos

Estados del usuario respecto a su pertenencia a la universidad o si tiene penalización:

- ACTIVO
- SUSPENDIDO
- INACTIVO
- BLOQUEADO

Campos

Campo	Tipo	Justificación
id_usuario	BIGINT	Identificador interno único
cedula	VARCHAR(10)	Documento oficial ecuatoriano
codigo_institucional	VARCHAR(20)	Identificador universitario
primer_nombre	VARCHAR(50)	Primer nombre del usuario
segundo_nombre	VARCHAR(50)	Segundo nombre del usuario
primer_apellido	VARCHAR(50)	Primero apellido del usuario
segundo_apellido	VARCHAR(50)	Segundo apellido del usuario

correo	VARCHAR(150)	Contacto institucional
telefono	VARCHAR(10)	Contacto adicional, considerar que para estandarización internacional se usa 15 carácteres
estado	VARCHAR(20)	Control de acceso bibliotecario
id_rol	BIGINT	Relación en como se aplica las reglas del préstamo

rol_usuario

Justificación: Permite definir reglas de préstamo distintas según el tipo de usuario.

Ejemplo de usuarios para préstamo del libro:

- estudiante
- docente
- administrativo

Ejemplo de reglas de préstamo

Rol	Máximo de libros	Días del préstamo
Estudiante	3	7
Docente	5	30
Administrativo	2	10

Pero eso depende de cada universidad. Por ello los valores son configurables

Campos

Campo	Tipo	Justificación
id_rol	BIGINT	Identificador interno
nombre	VARCHAR(50)	Nombre del rol
max_prestamos	INTEGER	Límite de libros prestados
dias_prestamo	INTEGER	Tiempo permitido en días

libro

Justificación: Representa la obra bibliográfica original

Campos

Campo	Tipo	Justificación
id_libro	BIGINT	Identificador interno
isbn	VARCHAR(13)	Código internacional ISBN, se almacena sin guiones para facilitar validaciones y búsquedas.
titulo	VARCHAR(200)	Nombre del libro
anio_publicacion	INTEGER	Año de publicación
descripcion	TEXT	Información adicional

id_categoria	BIGINT	Clasificación
id_editorial	BIGINT	Editorial asociada

¿Por qué ISBN no es id?

- hay libros antiguos sin ISBN
- algunos libros universitarios internos no tienen ISBN

categoria

Justificación: Permite clasificar libros según áreas temáticas para facilitar búsquedas y organización.

Ejemplos:

- programación
- matemáticas
- física

Campos

Campo	Tipo	Justificación
id_categoria	BIGINT	Identificador interno
nombre	VARCHAR(100)	Nombre categoría
descripcion	TEXT	Información adicional

editorial

Justificación: Representa la organización responsable de la publicación del libro.

Campos

Campo	Tipo	Justificación
id_editorial	BIGINT	Identificador
nombre	VARCHAR(100)	Nombre editorial
pais	VARCHAR(50)	País editorial

autor

Justificación: Representa a las personas responsables de la creación intelectual de una obra bibliográfica.

Campos

Campo	Tipo	Justificación
id_autor	BIGINT	Identificador interno
primer_nombre	VARCHAR(50)	Primer nombre del autor
segundo_nombre	VARCHAR(50)	Segundo nombre del autor
primer_apellido	VARCHAR(50)	Primero apellido del autor
segundo_apellido	VARCHAR(50)	Segundo apellido del autor

nacionalidad	VARCHAR(50)	Información bibliográfica

libro_autor

Justificación: Permite modelar la relación muchos a muchos entre libros y autores.

Campos

Campo	Tipo	Justificación
id_libro	BIGINT	Libro relacionado
id_autor	BIGINT	Autor relacionado

ejemplar

Justificación: Representa una copia física real de un libro. Esto es importante porque un libro puede tener múltiples copias físicas.

En sistemas reales de bibliotecas no se presta la obra bibliográfica general, sino una copia física específica.

Esto permite:

- controlar disponibilidad individual
- registrar pérdidas o daños
- gestionar múltiples copias del mismo libro
- mantener trazabilidad del inventario físico

Ejemplo:

Un libro puede tener 5 copias físicas, donde:

- 2 están prestadas
- 1 está dañada
- 2 disponibles

Sin la entidad ejemplar no sería posible representar correctamente este comportamiento.

Cada ejemplar puede tener estados distintos:

- DISPONIBLE
- PRESTADO
- PERDIDO
- DANIADO
- MANTENIMIENTO

Campos

Campo	Tipo	Justificación
id_ejemplar	BIGINT	Identificador interno
codigo_barra	VARCHAR(30)	Identificación física
id_libro	BIGINT	Libro asociado
estado	VARCHAR(20)	Estado del ejemplar
ubicacion_fisica	VARCHAR(100)	Estantería o sección
fecha_adquisicion	DATE	Control de inventario

prestamo

Justificación: Registra cuándo un usuario solicita un ejemplar, se presta un ejemplar no un libro

Un ejemplar solamente puede tener un préstamo ACTIVO al mismo tiempo.

Campos

Campo	Tipo	Justificación
id_prestamo	BIGINT	Identificador
id_usuario	BIGINT	Usuario solicitante
id_usuario_sistema	BIGINT	Usuario del sistema que registró prestamo
id_ejemplar	BIGINT	Ejemplar prestado
fecha_prestamo	TIMESTAMP	Inicio del préstamo
fecha_devolucion_esperada	TIMESTAMP	Fecha límite
fecha_devolucion_real	TIMESTAMP	Fecha real
estado	VARCHAR(20)	Estado del préstamo: ● ACTIVO ● DEVUELTO ● ATRASADO ● CANCELADO

sancion

Justificación: Permite registrar penalizaciones por incumplimientos.

Ejemplos reales:

- devolución tardía

- pérdida del libro
- daño físico

No toda sanción tiene penalización monetaria

Campos:

Campo	Tipo	Justificación
id_sancion	BIGINT	Identificador
id_usuario	BIGINT	Usuario sancionado
id_usuario_sistema	BIGINT	Usuario del sistema que registró la sanción
id_prestamo	BIGINT	Préstamo asociado
tipo	VARCHAR(30)	Clasificación
monto	NUMERIC(10,2)	Valor monetario, se utiliza NUMERIC en lugar de FLOAT para evitar errores de precisión en valores monetarios.
motivo	TEXT	Descripción
estado	VARCHAR(20)	Situación actual ● PENDIENTE ● PAGADA ● ACTIVA

		• CONDONADA (Perdonar sanción)
fecha_inicio	DATE	Inicio sanción
fecha_fin	DATE	Fin sanción

reserva

Justificación: Permite apartar un libro cuando no existen ejemplares disponibles.

Ejemplo real:

1. Todos los ejemplares están prestados
2. Usuario realiza reserva
3. Cuando un ejemplar se libera:
4. se notifica al usuario
5. Si no lo retira, la reserva expira

Campos

Campo	Tipo	Justificación
id_reserva	BIGINT	Identificador
id_usuario	BIGINT	Usuario solicitante
id_usuario_sistema	BIGINT	Usuario del sistema que registró reserva
id_libro	BIGINT	Libro reservado
fecha_reserva	TIMESTAMP	Momento reserva

fecha_expiracion	TIMESTAMP	Límite retiro
estado	VARCHAR(20)	Estado reserva: ● ACTIVA ● EXPIRADA ● COMPLETADA ● CANCELADA

auditoria

Justificación: Permite registrar acciones realizadas dentro del sistema.

Ejemplo de guardado de actividad:

Usuario sistema: bibliotecario01

Acción: UPDATE

Tabla:prestamo

Campos

Campo	Tipo	Justificación
id_auditoria	BIGINT	Identificador
tabla_afectada	VARCHAR(50)	Tabla modificada
id_registro	BIGINT	Registro afectado, no funciona como FK ya que no es solo a una tabla sino a varias
accion	VARCHAR(20)	INSERT/UPDATE/DELETE

usuario_sistema	VARCHAR(50)	Usuario responsable
fecha_evento	TIMESTAMP	Momento acción
valor_anterior	TEXT	Estado previo
valor_nuevo	TEXT	Estado nuevo

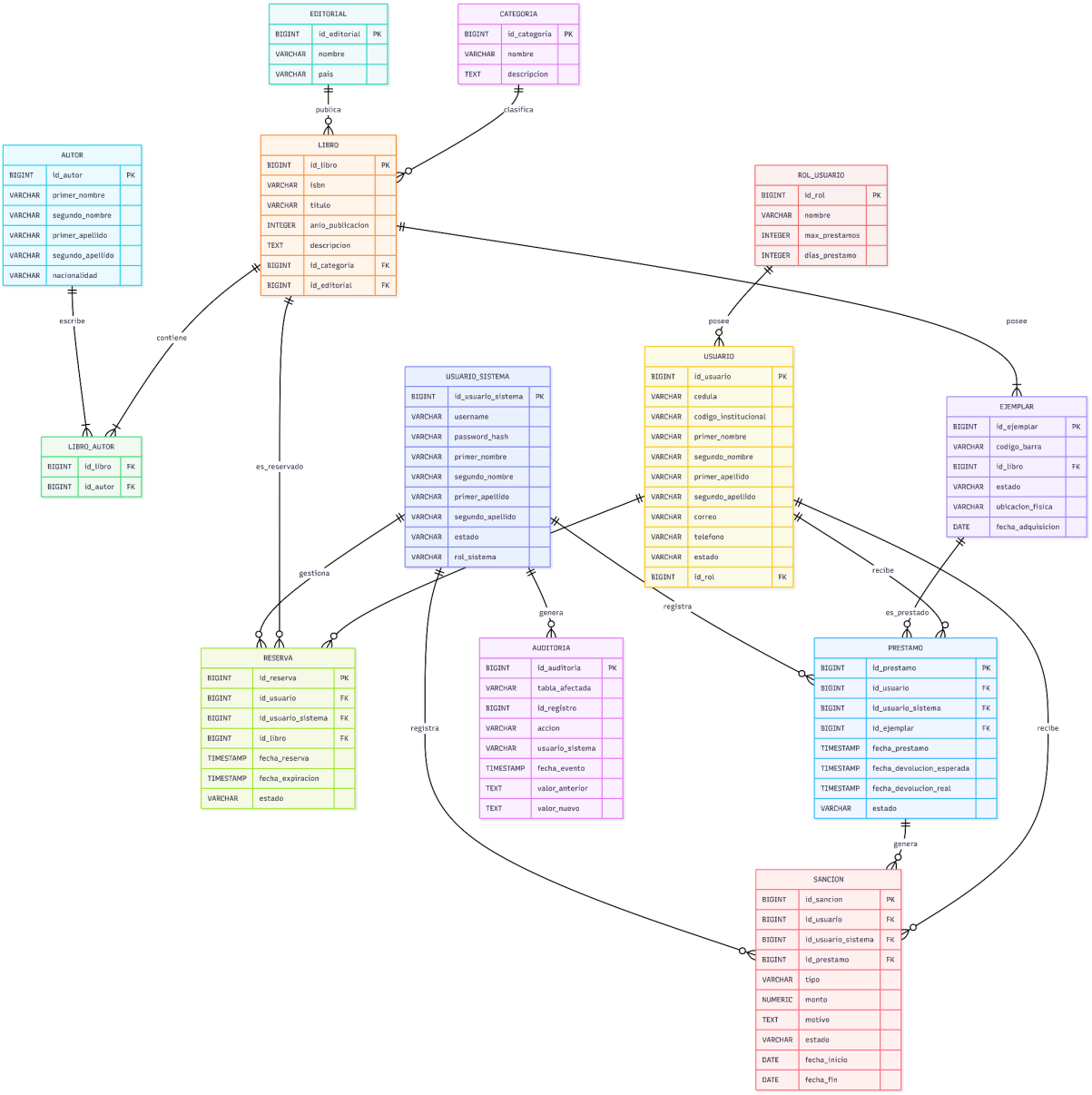
5.2 Relaciones

Tabla de relaciones

Relación	Cardinalidad
rol_usuario → usuario	1:N
categoria → libro	1:N
editorial → libro	1:N
libro ↔ autor	N:N
libro → libro_autor	1:N
autor → libro_autor	1:N

libro → ejemplar	1:N
usuario → prestamo	1:N
ejemplar → prestamo	1:N
usuario → sancion	1:N
prestamo → sancion	1:N
usuario → reserva	1:N
libro → reserva	1:N
usuario_sistema → auditoria	1:N
usuario_sistema → prestamo	1:N
usuario_sistema → reserva	1:N
usuario_sistema → sancion	1:N

5.3 Modelado



5.4 Script

```
-- ==========================================================
-- TABLA: rol_usuario
-- ==========================================================

CREATE TABLE rol_usuario (
    id_rol BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL UNIQUE,
    max_prestamos INTEGER NOT NULL CHECK (max_prestamos > 0),
    dias_prestamo INTEGER NOT NULL CHECK (dias_prestamo > 0)
);

-- ==========================================================
```

```sql
-- TABLA: usuario_sistema
-- ==========================================================

CREATE TABLE usuario_sistema (
    id_usuario_sistema BIGINT GENERATED ALWAYS AS IDENTITY
PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    primer_nombre VARCHAR(50) NOT NULL,
    segundo_nombre VARCHAR(50),
    primer_apellido VARCHAR(50) NOT NULL,
    segundo_apellido VARCHAR(50),
    estado VARCHAR(20) NOT NULL,
    rol_sistema VARCHAR(30) NOT NULL,

    CONSTRAINT chk_usuario_sistema_estado
    CHECK (estado IN ('ACTIVO', 'INACTIVO', 'BLOQUEADO')),

    CONSTRAINT chk_usuario_sistema_rol
    CHECK (rol_sistema IN ('ADMINISTRADOR', 'BIBLIOTECARIO'))
);

-- ==========================================================
-- TABLA: usuario
-- ==========================================================

CREATE TABLE usuario (
    id_usuario BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY
KEY,
    cedula VARCHAR(10),
    codigo_institucional VARCHAR(20) NOT NULL UNIQUE,
    primer_nombre VARCHAR(50) NOT NULL,
    segundo_nombre VARCHAR(50),
    primer_apellido VARCHAR(50) NOT NULL,
    segundo_apellido VARCHAR(50),
    correo VARCHAR(150) NOT NULL UNIQUE,
    telefono VARCHAR(15),
    estado VARCHAR(20) NOT NULL,
    id_rol BIGINT NOT NULL,

    CONSTRAINT fk_usuario_rol
    FOREIGN KEY (id_rol)
    REFERENCES rol_usuario(id_rol),

    CONSTRAINT uq_usuario_cedula
    UNIQUE (cedula),

    CONSTRAINT chk_usuario_estado
    CHECK (estado IN ('ACTIVO', 'SUSPENDIDO', 'INACTIVO',
'BLOQUEADO'))
);

-- ==========================================================
```

```sql
-- TABLA: categoria
-- ============================================================

CREATE TABLE categoria (
    id_categoria BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY
KEY,
    nombre VARCHAR(100) NOT NULL UNIQUE,
    descripcion TEXT
);

-- ============================================================
-- TABLA: editorial
-- ============================================================

CREATE TABLE editorial (
    id_editorial BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY
KEY,
    nombre VARCHAR(100) NOT NULL UNIQUE,
    pais VARCHAR(50)
);

-- ============================================================
-- TABLA: libro
-- ============================================================

CREATE TABLE libro (
    id_libro BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    isbn VARCHAR(13),
    titulo VARCHAR(200) NOT NULL,
    anio_publicacion INTEGER,
    descripcion TEXT,
    id_categoria BIGINT NOT NULL,
    id_editorial BIGINT NOT NULL,

    CONSTRAINT fk_libro_categoria
    FOREIGN KEY (id_categoria)
    REFERENCES categoria(id_categoria),

    CONSTRAINT fk_libro_editorial
    FOREIGN KEY (id_editorial)
    REFERENCES editorial(id_editorial),

    CONSTRAINT uq_libro_isbn
    UNIQUE (isbn),

    CONSTRAINT chk_libro_anio
    CHECK (
        anio_publicacion IS NULL
        OR anio_publicacion >= 1400
    )
);

-- ============================================================
```

```sql
-- TABLA: autor
-- ==========================================================

CREATE TABLE autor (
    id_autor BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    primer_nombre VARCHAR(50) NOT NULL,
    segundo_nombre VARCHAR(50),
    primer_apellido VARCHAR(50) NOT NULL,
    segundo_apellido VARCHAR(50),
    nacionalidad VARCHAR(50)
);

-- ==========================================================
-- TABLA: libro_autor
-- ==========================================================

CREATE TABLE libro_autor (
    id_libro BIGINT NOT NULL,
    id_autor BIGINT NOT NULL,

    PRIMARY KEY (id_libro, id_autor),

    CONSTRAINT fk_libro_autor_libro
    FOREIGN KEY (id_libro)
    REFERENCES libro(id_libro),

    CONSTRAINT fk_libro_autor_autor
    FOREIGN KEY (id_autor)
    REFERENCES autor(id_autor)
);

-- ==========================================================
-- TABLA: ejemplar
-- ==========================================================

CREATE TABLE ejemplar (
    id_ejemplar BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY
KEY,
    codigo_barra VARCHAR(30),
    id_libro BIGINT NOT NULL,
    estado VARCHAR(20) NOT NULL,
    ubicacion_fisica VARCHAR(100),
    fecha_adquisicion DATE,

    CONSTRAINT fk_ejemplar_libro
    FOREIGN KEY (id_libro)
    REFERENCES libro(id_libro),

    CONSTRAINT uq_ejemplar_codigo_barra
    UNIQUE (codigo_barra),

    CONSTRAINT chk_ejemplar_estado
```

```sql
    CHECK (
        estado IN (
            'DISPONIBLE',
            'PRESTADO',
            'PERDIDO',
            'DANIADO',
            'MANTENIMIENTO'
        )
    )
);

-- ============================================================
-- TABLA: prestamo
-- ============================================================

CREATE TABLE prestamo (
    id_prestamo BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY
KEY,
    id_usuario BIGINT NOT NULL,
    id_usuario_sistema BIGINT NOT NULL,
    id_ejemplar BIGINT NOT NULL,
    fecha_prestamo TIMESTAMP NOT NULL DEFAULT
CURRENT_TIMESTAMP,
    fecha_devolucion_esperada TIMESTAMP NOT NULL,
    fecha_devolucion_real TIMESTAMP,
    estado VARCHAR(20) NOT NULL,

    CONSTRAINT fk_prestamo_usuario
    FOREIGN KEY (id_usuario)
    REFERENCES usuario(id_usuario),

    CONSTRAINT fk_prestamo_usuario_sistema
    FOREIGN KEY (id_usuario_sistema)
    REFERENCES usuario_sistema(id_usuario_sistema),

    CONSTRAINT fk_prestamo_ejemplar
    FOREIGN KEY (id_ejemplar)
    REFERENCES ejemplar(id_ejemplar),

    CONSTRAINT chk_prestamo_estado
    CHECK (
        estado IN (
            'ACTIVO',
            'DEVUELTO',
            'ATRASADO',
            'CANCELADO'
        )
    ),

    CONSTRAINT chk_fechas_prestamo
    CHECK (
        fecha_devolucion_esperada >= fecha_prestamo
    )
```

```sql
);

-- ==========================================================
-- TABLA: sancion
-- ==========================================================

CREATE TABLE sancion (
    id_sancion BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY
KEY,
    id_usuario BIGINT NOT NULL,
    id_usuario_sistema BIGINT NOT NULL,
    id_prestamo BIGINT NOT NULL,
    tipo VARCHAR(30) NOT NULL,
    monto NUMERIC(10,2) DEFAULT 0,
    motivo TEXT,
    estado VARCHAR(20) NOT NULL,
    fecha_inicio DATE NOT NULL,
    fecha_fin DATE,

    CONSTRAINT fk_sancion_usuario
    FOREIGN KEY (id_usuario)
    REFERENCES usuario(id_usuario),

    CONSTRAINT fk_sancion_usuario_sistema
    FOREIGN KEY (id_usuario_sistema)
    REFERENCES usuario_sistema(id_usuario_sistema),

    CONSTRAINT fk_sancion_prestamo
    FOREIGN KEY (id_prestamo)
    REFERENCES prestamo(id_prestamo),

    CONSTRAINT chk_sancion_estado
    CHECK (
        estado IN (
            'PENDIENTE',
            'PAGADA',
            'ACTIVA',
            'CONDONADA'
        )
    ),

    CONSTRAINT chk_sancion_monto
    CHECK (monto >= 0)
);

-- ==========================================================
-- TABLA: reserva
-- ==========================================================

CREATE TABLE reserva (
    id_reserva BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY
KEY,
    id_usuario BIGINT NOT NULL,
```

```sql
    id_usuario_sistema BIGINT NOT NULL,
    id_libro BIGINT NOT NULL,
    fecha_reserva TIMESTAMP NOT NULL DEFAULT
CURRENT_TIMESTAMP,
    fecha_expiracion TIMESTAMP NOT NULL,
    estado VARCHAR(20) NOT NULL,

    CONSTRAINT fk_reserva_usuario
    FOREIGN KEY (id_usuario)
    REFERENCES usuario(id_usuario),

    CONSTRAINT fk_reserva_usuario_sistema
    FOREIGN KEY (id_usuario_sistema)
    REFERENCES usuario_sistema(id_usuario_sistema),

    CONSTRAINT fk_reserva_libro
    FOREIGN KEY (id_libro)
    REFERENCES libro(id_libro),

    CONSTRAINT chk_reserva_estado
    CHECK (
        estado IN (
            'ACTIVA',
            'EXPIRADA',
            'COMPLETADA',
            'CANCELADA'
        )
    )
);

-- ==========================================================
-- TABLA: auditoria
-- ==========================================================

CREATE TABLE auditoria (
    id_auditoria BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY
KEY,
    tabla_afectada VARCHAR(50) NOT NULL,
    id_registro BIGINT NOT NULL,
    accion VARCHAR(20) NOT NULL,
    usuario_sistema VARCHAR(50) NOT NULL,
    fecha_evento TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    valor_anterior TEXT,
    valor_nuevo TEXT,

    CONSTRAINT chk_auditoria_accion
    CHECK (accion IN ('INSERT', 'UPDATE', 'DELETE'))
);

-- ==========================================================
-- INDICES
-- ==========================================================
```

```sql
CREATE INDEX idx_libro_titulo
ON libro(titulo);

CREATE INDEX idx_usuario_correo
ON usuario(correo);

CREATE INDEX idx_prestamo_estado
ON prestamo(estado);

CREATE INDEX idx_ejemplar_estado
ON ejemplar(estado);

CREATE INDEX idx_reserva_estado
ON reserva(estado);

-- ==========================================================
-- FUNCION:
-- VALIDAR QUE EL EJEMPLAR ESTE DISPONIBLE
-- ==========================================================

CREATE OR REPLACE FUNCTION fn_validar_ejemplar_disponible()
RETURNS TRIGGER AS $$
DECLARE
    estado_actual VARCHAR(20);
BEGIN

    SELECT estado
    INTO estado_actual
    FROM ejemplar
    WHERE id_ejemplar = NEW.id_ejemplar;

    IF estado_actual <> 'DISPONIBLE' THEN
        RAISE EXCEPTION
        'El ejemplar no se encuentra disponible para
préstamo';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_validar_ejemplar_disponible
BEFORE INSERT ON prestamo
FOR EACH ROW
EXECUTE FUNCTION fn_validar_ejemplar_disponible();

-- ==========================================================
-- FUNCION:
-- ACTUALIZAR ESTADO DEL EJEMPLAR
-- ==========================================================

CREATE OR REPLACE FUNCTION fn_actualizar_estado_ejemplar()
```

```
RETURNS TRIGGER AS $$
BEGIN

    IF NEW.estado = 'ACTIVO' THEN

        UPDATE ejemplar
        SET estado = 'PRESTADO'
        WHERE id_ejemplar = NEW.id_ejemplar;

    ELSIF NEW.estado = 'DEVUELTO' THEN

        UPDATE ejemplar
        SET estado = 'DISPONIBLE'
        WHERE id_ejemplar = NEW.id_ejemplar;

    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_actualizar_estado_ejemplar
AFTER INSERT OR UPDATE ON prestamo
FOR EACH ROW
EXECUTE FUNCTION fn_actualizar_estado_ejemplar();

-- ==========================================================
-- FUNCION:
-- VALIDAR LIMITE DE PRESTAMOS
-- ==========================================================

CREATE OR REPLACE FUNCTION fn_validar_limite_prestamos()
RETURNS TRIGGER AS $$
DECLARE
    prestamos_activos INTEGER;
    limite_permitido INTEGER;
BEGIN

    SELECT COUNT(*)
    INTO prestamos_activos
    FROM prestamo
    WHERE id_usuario = NEW.id_usuario
    AND estado = 'ACTIVO';

    SELECT ru.max_prestamos
    INTO limite_permitido
    FROM usuario u
    JOIN rol_usuario ru
        ON u.id_rol = ru.id_rol
    WHERE u.id_usuario = NEW.id_usuario;

    IF prestamos_activos >= limite_permitido THEN
```

```
        RAISE EXCEPTION
        'El usuario alcanzó el límite máximo de préstamos';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_validar_limite_prestamos
BEFORE INSERT ON prestamo
FOR EACH ROW
EXECUTE FUNCTION fn_validar_limite_prestamos();

-- ==========================================================
-- FUNCION:
-- EVITAR MULTIPLES PRESTAMOS ACTIVOS DEL MISMO EJEMPLAR
-- ==========================================================

CREATE OR REPLACE FUNCTION
fn_validar_prestamo_activo_ejemplar()
RETURNS TRIGGER AS $$
DECLARE
    cantidad INTEGER;
BEGIN

    SELECT COUNT(*)
    INTO cantidad
    FROM prestamo
    WHERE id_ejemplar = NEW.id_ejemplar
    AND estado = 'ACTIVO';

    IF cantidad > 0 THEN
        RAISE EXCEPTION
        'El ejemplar ya posee un préstamo activo';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_validar_prestamo_activo_ejemplar
BEFORE INSERT ON prestamo
FOR EACH ROW
EXECUTE FUNCTION fn_validar_prestamo_activo_ejemplar();

-- ==========================================================
-- FUNCION:
-- VALIDAR QUE SOLO SE RESERVE SI NO HAY DISPONIBLES
-- ==========================================================

CREATE OR REPLACE FUNCTION fn_validar_reserva()
RETURNS TRIGGER AS $$
DECLARE
```

```
    disponibles INTEGER;
BEGIN

    SELECT COUNT(*)
    INTO disponibles
    FROM ejemplar
    WHERE id_libro = NEW.id_libro
    AND estado = 'DISPONIBLE';

    IF disponibles > 0 THEN
        RAISE EXCEPTION
        'No se puede reservar porque existen ejemplares
disponibles';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_validar_reserva
BEFORE INSERT ON reserva
FOR EACH ROW
EXECUTE FUNCTION fn_validar_reserva();
```

5.5 Reglas de negocio implementadas

Validación de Disponibilidad

Un ejemplar únicamente puede prestarse si se encuentra en estado DISPONIBLE.

Límite de Préstamos

Cada usuario posee un máximo permitido de préstamos según su rol.

Exclusividad de Préstamo Activo

Un ejemplar no puede tener múltiples préstamos activos simultáneamente.

Reservas Condicionadas

Un libro solo puede reservarse si no existen ejemplares disponibles.

Integridad de Estados

Se utilizan restricciones CHECK para evitar valores inválidos.

Integridad Monetaria

Las sanciones monetarias no pueden ser negativas.

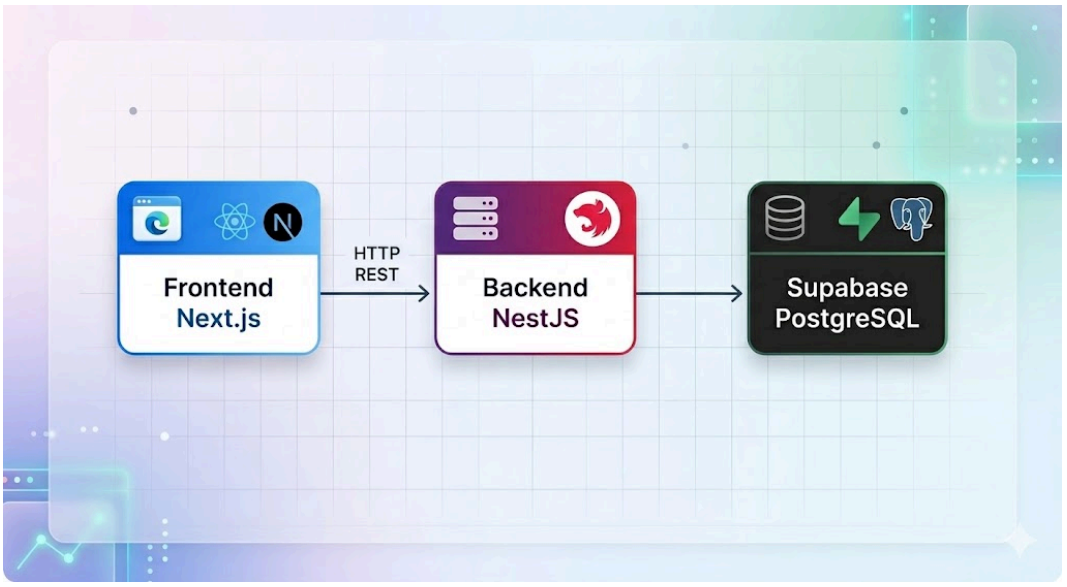
6. Arquitectura General del Sistema

6.1 Arquitectura Global

El sistema utiliza una arquitectura full-stack desacoplada basada en:

- frontend SPA,
- backend API REST,
- base de datos PostgreSQL.

La comunicación se realiza mediante HTTP y consumo de endpoints REST.



6.2. Arquitectura Backend

Tecnologías Utilizadas

- NestJS
- Node.js
- TypeScript
- Supabase
- PostgreSQL
- Jest
- Supertest
- ESLint
- Prettier

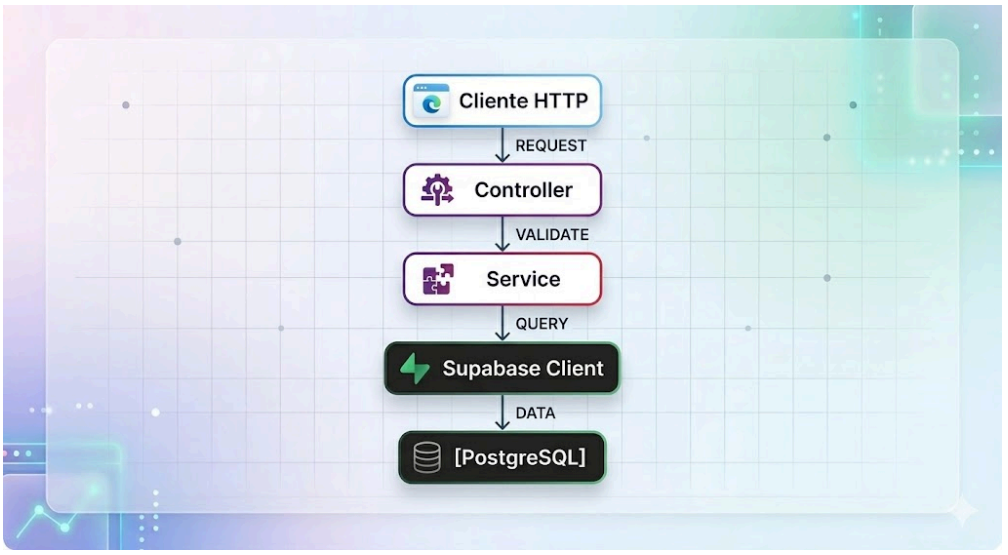
Patrón Arquitectónico

El backend utiliza arquitectura modular basada en NestJS.

Componentes:

- módulos,
- controladores,
- servicios,
- DTOs,
- entidades.

Flujo Backend



Responsabilidades

Controllers

Gestionan:

- rutas,
- endpoints,
- respuestas HTTP.

Services

Implementan:

- lógica de negocio,
- validaciones,
- reglas operativas.

DTOs

Permiten:

- validación,
- tipado,
- transferencia segura de datos.

Entities

Representan modelos del dominio.

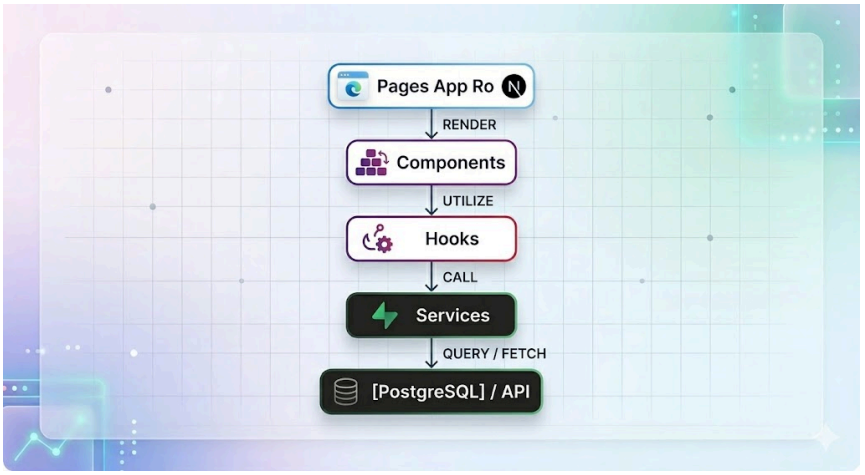
6.3. Arquitectura Frontend

Tecnologías Utilizadas

- Next.js
- React 19
- TypeScript
- Tailwind CSS
- Supabase JS

Arquitectura

El frontend sigue una arquitectura basada en componentes y App Router.



Características

- Características
- renderizado híbrido,
- componentes reutilizables,
- tipado fuerte,
- separación modular,
- consumo de API REST.

7. Testing y Calidad del Software

7.1. Estrategia de Calidad

El proyecto implementa mecanismos de calidad continua orientados a:

- reducción de defectos,
- mantenibilidad,
- cobertura funcional,
- análisis estático,
- detección temprana de errores.

7.2. Jest

Jest se utiliza para pruebas unitarias y pruebas de integración.

Objetivos

- validar lógica de negocio,
- detectar regresiones,
- asegurar el comportamiento esperado.

Tipos de pruebas

Pruebas Unitarias

Validan:

- servicios,
- funciones,
- reglas de negocio.

7.3. Supertest

Supertest se utiliza para pruebas E2E sobre endpoints HTTP.

Validaciones

- códigos HTTP,
- respuestas JSON,
- comportamiento API,
- validaciones de entrada.

7.4. SonarQube

SonarQube permite realizar análisis estático continuo del proyecto.

Objetivos

- detectar code smells,
- detectar vulnerabilidades,
- evaluar cobertura,
- controlar deuda técnica.

Métricas Monitoreadas

- cobertura de pruebas,
- duplicación de código,
- complejidad ciclomática,
- vulnerabilidades,
- mantenibilidad.

7.5. ESLint

ESLint se utiliza para mantener consistencia y calidad del código.

Objetivos

- estandarización,
- detección de errores,
- buenas prácticas.

8. Seguridad y Validaciones

El sistema implementa:

- validación de DTOs,
- validación SQL,
- restricciones CHECK,
- control de integridad,
- validación mediante triggers,
- almacenamiento seguro de contraseñas.

9. Tradeoffs Técnicos

9.1. Uso de Supabase

Ventajas:

- rápida integración,
- PostgreSQL administrado,
- facilidad de despliegue.

Desventajas:

- dependencia de proveedor externo,
- limitaciones según el plan.

9.2. Arquitectura Monolítica Modular

Ventajas:

- simplicidad inicial,
- menor complejidad operativa,

- facilidad de mantenimiento académico.

Desventajas:

- menor escalabilidad que microservicios.

10. Conclusiones

- El sistema web de gestión bibliotecaria permitió implementar una solución orientada a procesos reales del dominio bibliotecario universitario, integrando funcionalidades para administración de libros, ejemplares, préstamos, reservas, sanciones y auditoría. La utilización de una arquitectura modular y tecnologías modernas facilitó la organización del proyecto y la separación de responsabilidades entre frontend, backend y base de datos. Como resultado, se obtuvo una aplicación estructurada, mantenible y preparada para futuras ampliaciones.

- El modelado de la base de datos permitió representar correctamente las entidades y relaciones del sistema bibliotecario mediante PostgreSQL y Supabase. La incorporación de claves foráneas, restricciones CHECK, índices, triggers y funciones PL/pgSQL fortaleció la integridad de la información y el cumplimiento de reglas operativas críticas. Esto permitió mantener consistencia entre el modelo de datos y el comportamiento funcional esperado del sistema.

- La implementación del backend con NestJS y del frontend con Next.js permitió desarrollar una aplicación desacoplada y organizada por componentes y módulos funcionales. El uso de TypeScript contribuyó a mejorar el tipado, reducir errores y facilitar el mantenimiento del código. En consecuencia, la arquitectura adoptada favorece la escalabilidad y reutilización de componentes.

- La incorporación de pruebas automatizadas mediante Jest y Supertest permitió validar reglas de negocio, comportamiento de endpoints y funcionamiento de procesos críticos del sistema. Estas pruebas contribuyeron a detectar errores tempranamente y mejorar la confiabilidad de la aplicación durante el desarrollo. Por ello, el proyecto fortaleció su calidad funcional mediante mecanismos de validación automatizada.

- La aplicación de herramientas de análisis estático como ESLint y SonarQube permitió establecer estándares de codificación y detectar problemas relacionados con mantenibilidad, complejidad y calidad del software. El uso de estas herramientas facilitó la reducción de deuda técnica y promovió buenas prácticas de desarrollo. De esta manera, el proyecto incorporó mecanismos orientados a la mejora continua de la calidad del código.

11. Recomendaciones

- Se recomienda ampliar progresivamente la cobertura de pruebas automatizadas, priorizando módulos críticos como autenticación, préstamos, reservas y sanciones. La implementación de más pruebas unitarias, E2E e incluir pruebas de integración permitirá validar escenarios complejos y reducir riesgos de regresión ante futuras modificaciones. Esto contribuirá a incrementar la estabilidad y confiabilidad general del sistema.

- Es recomendable integrar procesos de integración continua (CI/CD) que automaticen la ejecución de pruebas, validaciones ESLint y análisis SonarQube antes de despliegues o integraciones de código. La automatización de verificaciones permitirá detectar errores de manera temprana y mantener estándares de calidad constantes durante el ciclo de desarrollo. Como resultado, se mejorará el control técnico y la mantenibilidad del proyecto.

- Se recomienda mantener actualizada la documentación técnica, las historias de usuario y los criterios de aceptación conforme evolucione el proyecto. La sincronización entre requisitos funcionales, reglas de negocio y pruebas automatizadas facilitará futuras tareas de mantenimiento y ampliación del sistema. De esta manera, se garantizará coherencia entre documentación, implementación y validaciones técnicas.